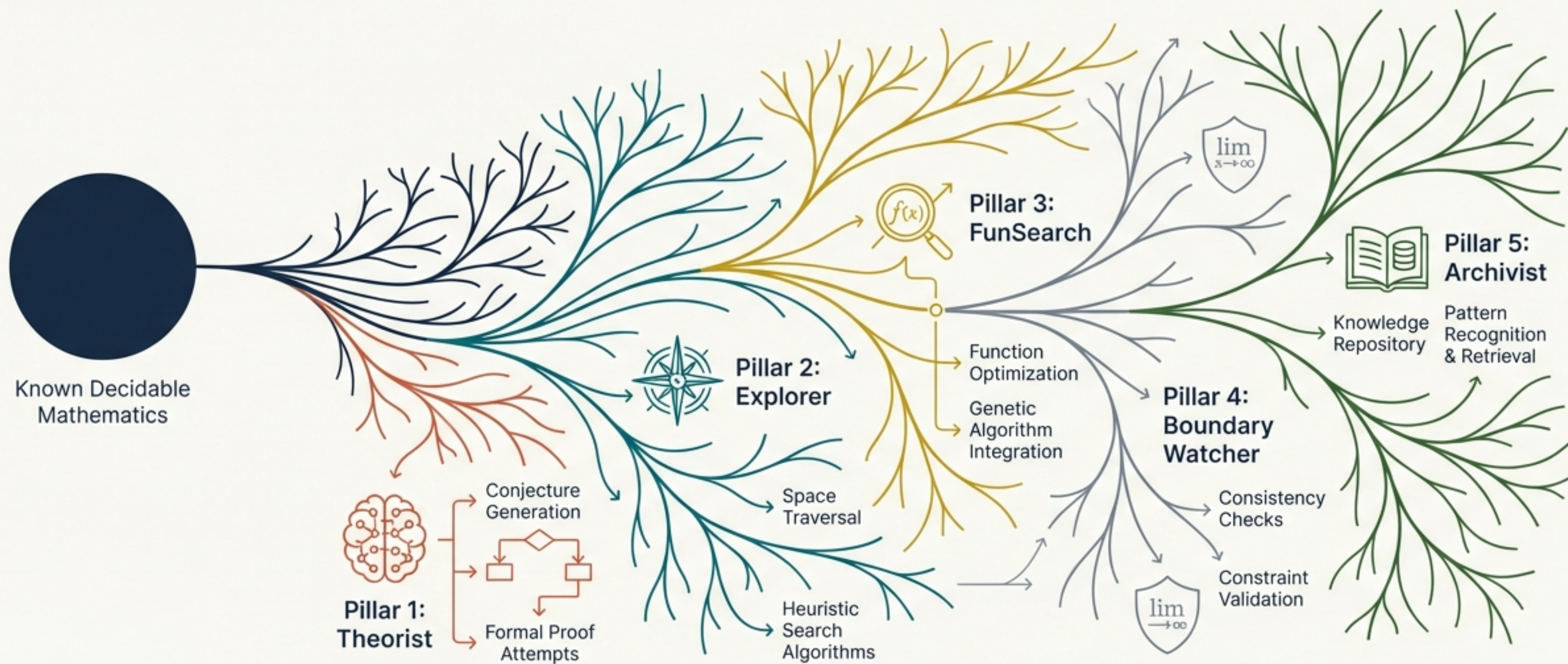


# Phase 6: Architecting the Mathematical Discovery Engine

From a Problem-Solving Tool to an Autonomous Research Associate



# A Necessary Evolution: From Efficiency to Exploration

## Phases 1-5: The Problem-Solving Engine



### Focus

Mastering **decidable** and **computable** mathematics.



### Function

Answering known user queries with high reliability and efficiency.

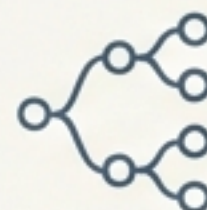


### Analogy

A highly advanced calculator or expert system.



## Phase 6: The Discovery Engine



### Focus

Confronting the 'Hard Limits'—**undecidability** and the **infinite search space** of open problems.



### Function

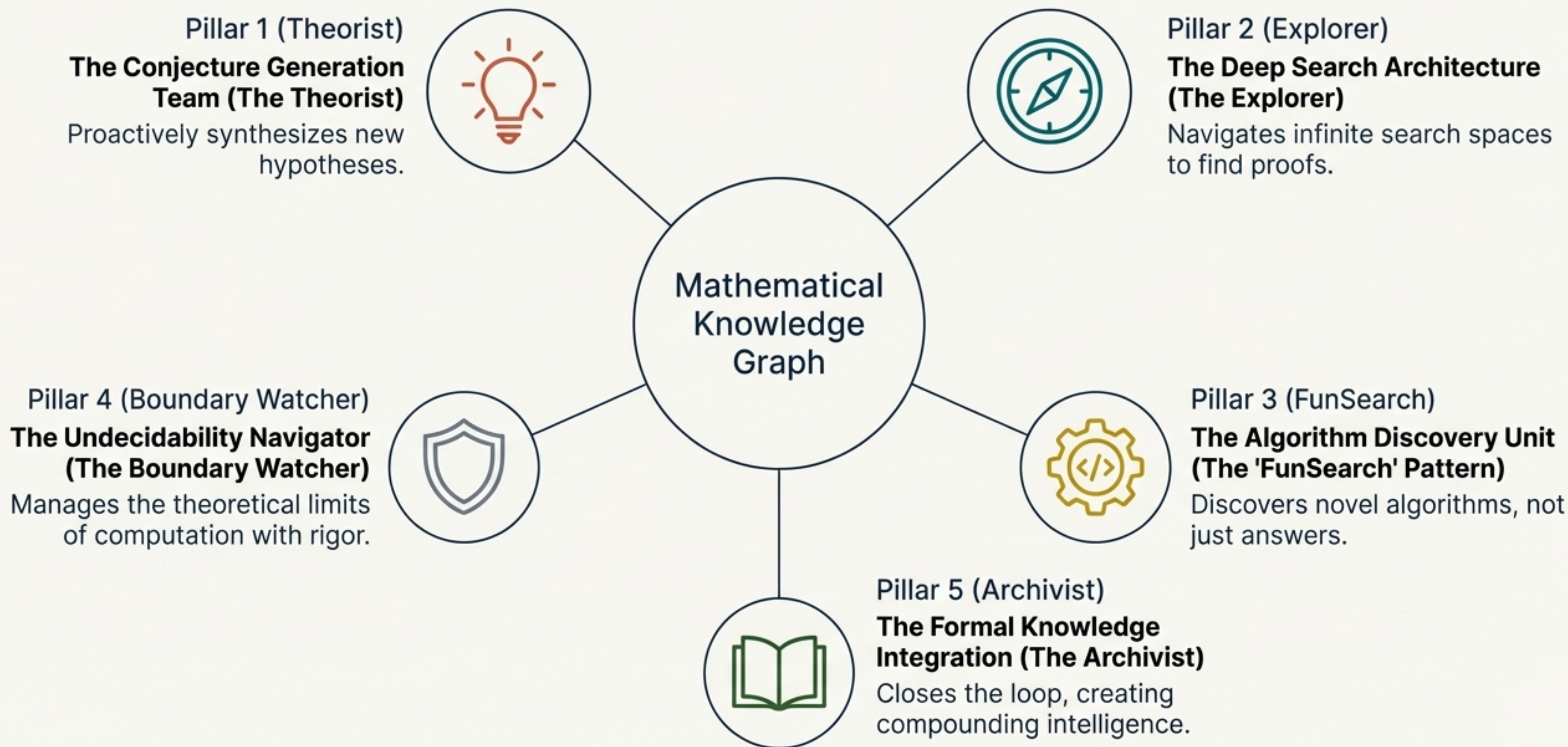
Generating new mathematical knowledge and expanding formalized mathematics.



### Analogy

An autonomous researcher.

# The Five Pillars of the Discovery Engine



# Pillar 1: The Theorist

## From Retrieval to Proactive Synthesis

### Core Objective

Upgrade the reactive Hypothesis Generator (Phase 3) into a proactive Conjecture Engine.

### Key Question It Answers

Instead of "What is the answer to this query?", it asks, "**What *should* be true given what we know?**"

### Mechanism Overview

1. Autonomously scans the Knowledge Graph to identify “holes” or patterns in existing theorems.
2. Uses a synthetic data generation pipeline, inspired by **AlphaGeometry**, to create a massive corpus of potential relationships.
3. Filters this corpus to propose novel, non-trivial lemmas and candidate conjectures.

# The Theorist's Pipeline: How a Conjecture is Born

## Stage 1: The Dreamer (Synthetic Data Generator)



Continuously generates millions of "synthetic theorems" by randomly sampling premises and deriving conclusions.

Creates a vast dataset of premise-conclusion pairs.

**Output:** 100M+ unique, logically true (but often trivial) statements.

## Stage 2: The Filter (Pattern Recognizer)



Sifts through the synthetic theorems.

Uses heuristics to filter out tautologies (e.g.,  $x=x$ ) and identify "interesting," non-trivial relationships that appear frequently.

**Output:** A curated list of "Candidate Conjectures."

## Stage 3: The Conjecture Formalizer



Translates the candidate conjectures into rigorous mathematical language.

Prepares the raw relationships as formal Lean or Isabelle statements for verification attempts.

**Output:** Formally-stated conjectures ready for the Deep Search Team.

# Pillar 2: The Explorer

## Navigating Infinite Search Spaces

### Core Objective

To find proof paths for open problems (like the IMO Grand Challenge) where the solution is completely unknown.

### The Technical Challenge

Standard tree-based reasoning (e.g., “Tree-of-Thoughts”) is insufficient for the combinatorial explosion of possibilities in open-ended mathematical search.

### The Architectural Solution

A **Product-Node Search Tree** architecture from **AlphaProof**, guided by Reinforcement Learning.

### Key Innovation

Uses a **Value Function** to estimate the probability that a specific tactical branch will lead to a proof, allowing for intelligent pruning of the search space.

# The Explorer's Engine: An RL-Guided Search Architecture

## Component 1: The Policy Network (The 'Tactician')

**Role:** "The 'Prover'".

**Function:** Generates a massive search tree of potential proof steps (tactics). It operates probabilistically, suggesting the most promising "next move" in a proof based on learned patterns.

## Component 2: The Critic Network (The 'Evaluator')

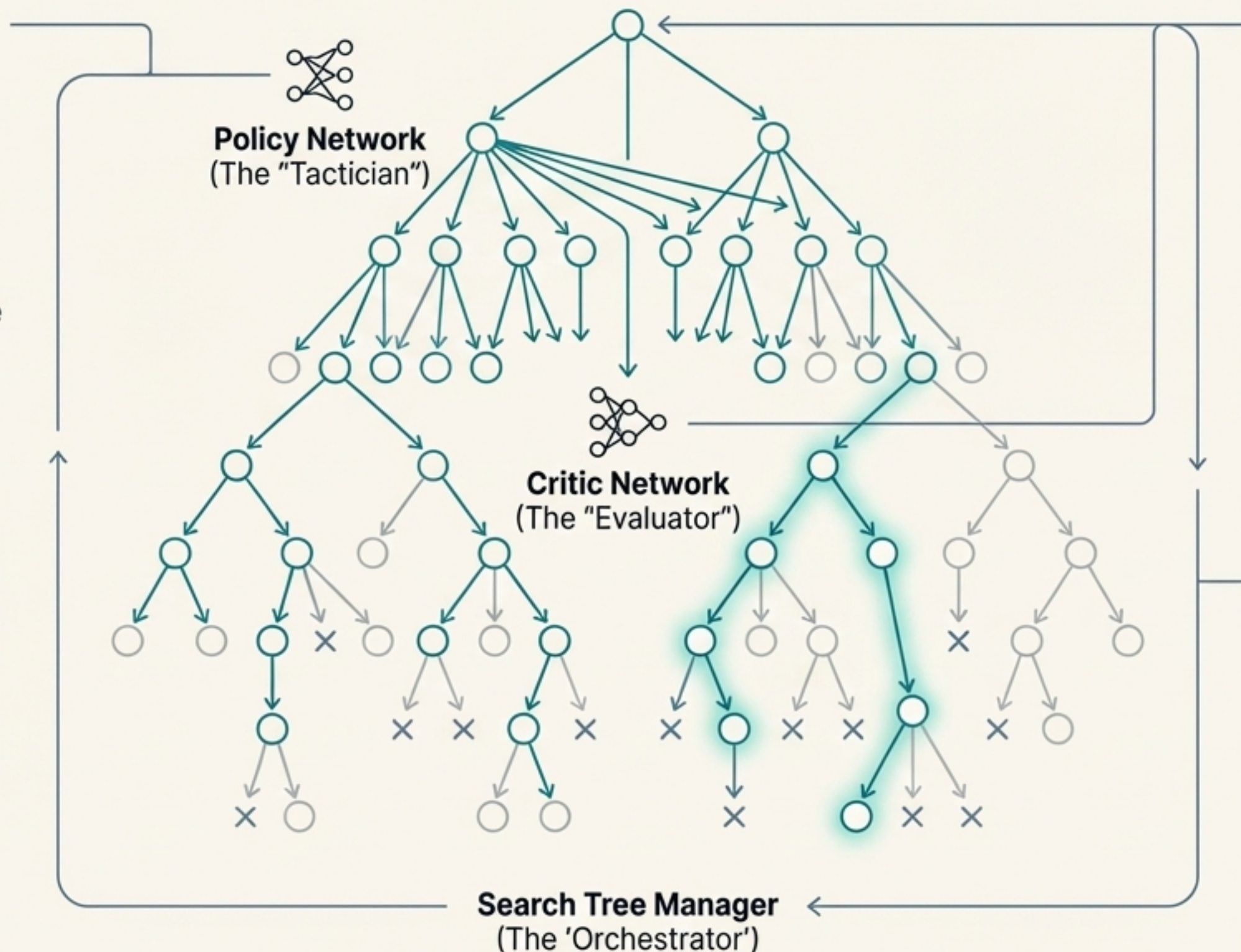
**Role:** "The 'Value-Function Agent'".

**Function:** Estimates the promise of each branch *without* fully expanding it. This allows the system to prune dead ends early, managing the computational explosion.

## Component 3: The Search Tree Manager

**Role:** The 'Orchestrator'.

**Function:** Manages thousands of parallel proof attempts, prioritizing branches deemed promising by the Critic.



# Pillar 3: The FunSearch Pattern

## Discovering Algorithms, Not Just Answers

Q.E.D.

Paradigm Shift

```
def search_for_algorithm(problem):  
    for attempt in range(MAX_ATTEMPTS):  
        candidate_function = generate_candidate(problem)  
        if evaluate(candidate_function) is True:  
            return candidate_function  
    return None
```

### Core Objective

Establish a specialized unit that searches for a **Python/Lean function**, not a number or a proof.

### Strategic Impact

- Unlocks progress in areas notoriously difficult for standard solvers, such as **Combinatorics**.
- Allows the system to discover new, verifiable algorithmic approaches rather than relying on rote memorization or existing theorems.

**Example Problem:** “Find a more efficient algorithm for bin packing” or “Discover a faster matrix multiplication algorithm.”

# The FunSearch Workflow: An Evolutionary Loop for Code

## 1. Propos Sandbox

**Agent:** An LLM-based agent.

**Action:** Generates Python or Julia code snippets representing a heuristic. Uses evolutionary logic, mutating the best code from the previous generation to find optimizations.

## 3. Evolver (Feedback Loop)

**Agent:** The core control logic.

**Action:** Feeds the scores and successful code snippets back to the Proposer to inform the next generation of code.

## 1. Proposer (Code Evolutionary Proposer)



### Final Output: The Heuristic Distiller

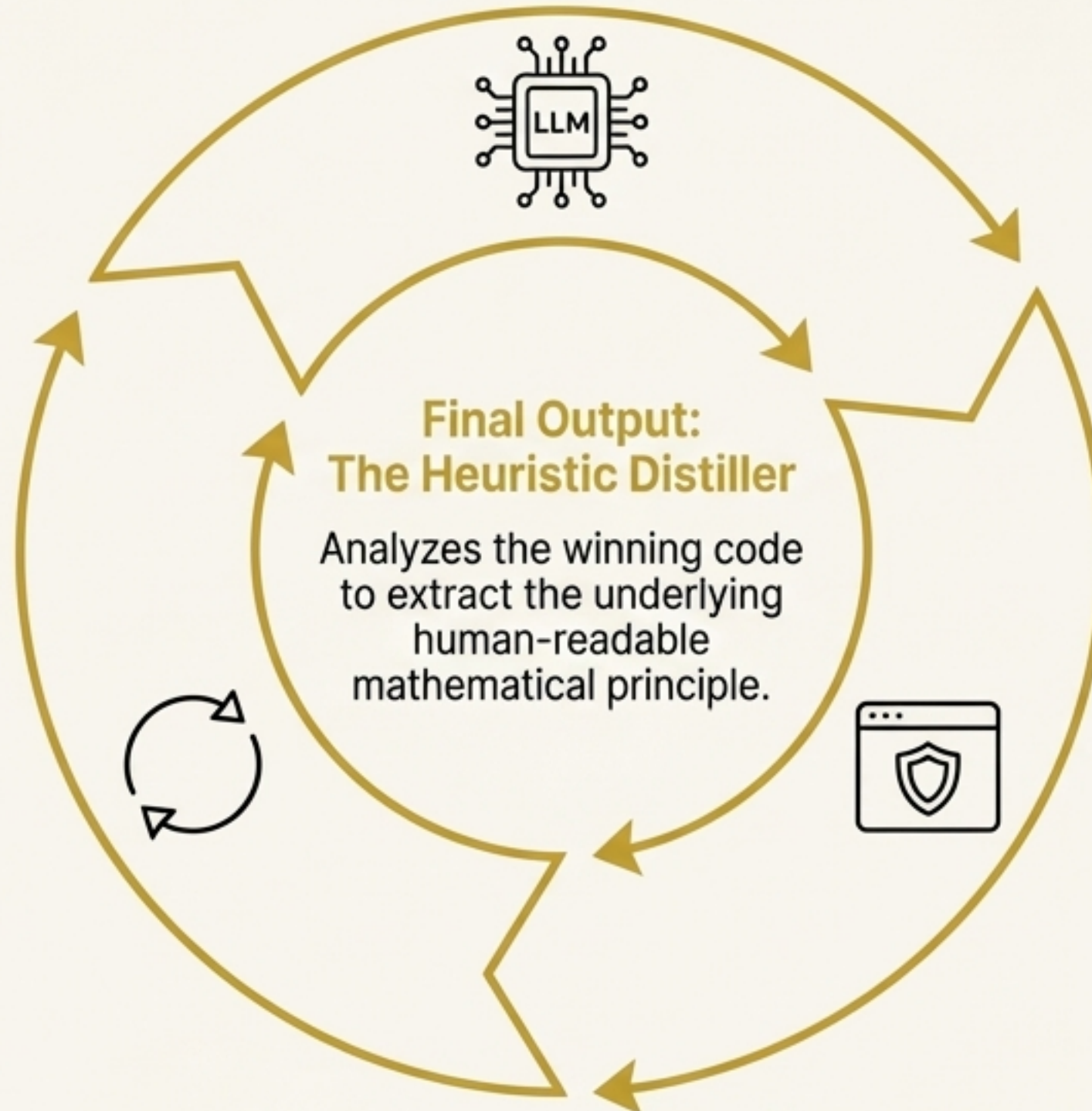
Analyzes the winning code to extract the underlying human-readable mathematical principle.



## 2. Evaluator (Sandbox Evaluator)

**Agent:** A secure execution environment.

**Action:** Runs the proposed code against a rigorous test set and returns a scalar score (e.g., speed, efficiency). Critically, it filters out incorrect code.



# Pillar 4: The Boundary Watcher

## Acknowledging the Hard Limits of Mathematics



### Core Objective

Protect the system from Gödel's Incompleteness and the Halting Problem by recognizing undecidable problems.

### The Strategic Imperative

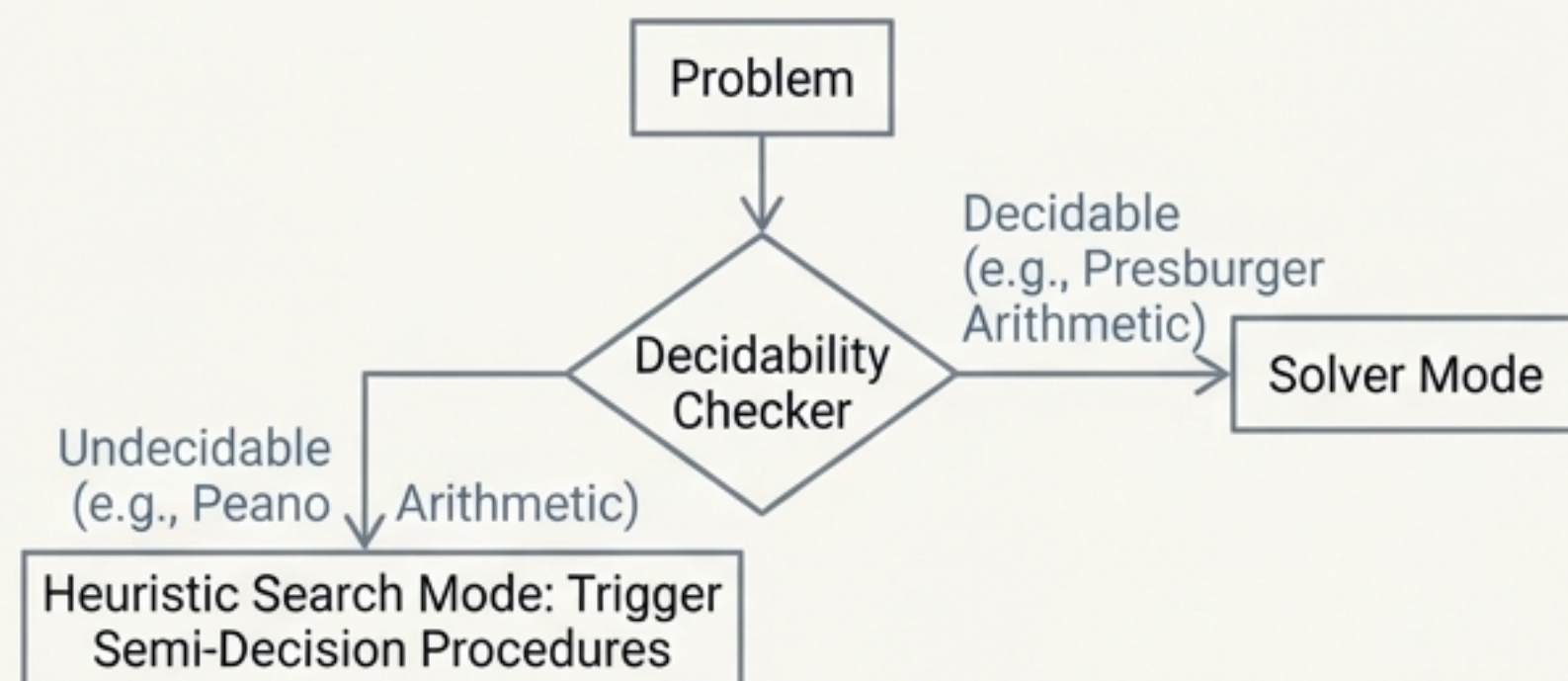
An autonomous research system *\*must\** know the boundaries of its own logic. This is not a limitation, but a crucial safety feature to prevent hallucination and ensure it remains a trusted tool for high-stakes inquiry.

### The Protocol

When the system encounters a problem in an undecidable domain (e.g., Diophantine Equations, Topology), the Decidability Checker flags it to switch from "Solver Mode" to "Heuristic Search Mode."

# The Boundary Watcher's Protocol for Undecidable Domains

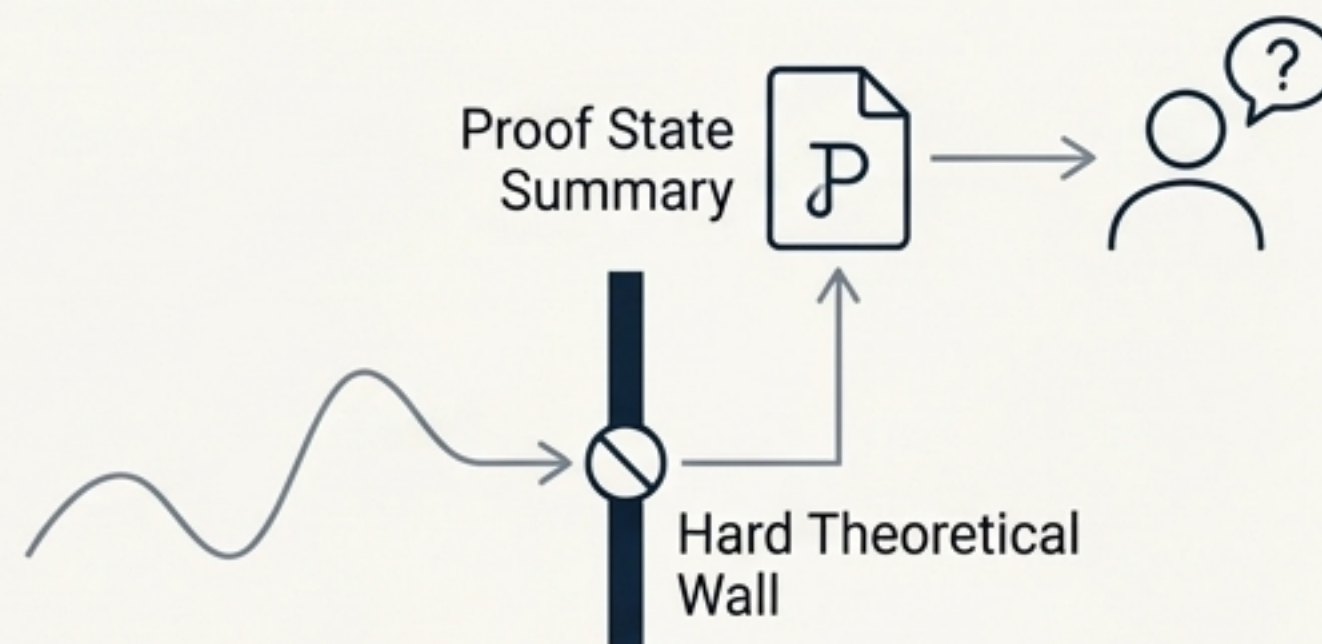
## Part 1: Automated Heuristic Search



**Agent:** Decidability Checker.

**Action:** For undecidable problems, it triggers Semi-Decision Procedures, which search for counter-examples or approximate solutions instead of attempting a direct proof that may loop forever.

## Part 2: Interactive Human-in-the-Loop Guidance



**Agent:** Interactive Guidance Liaison.

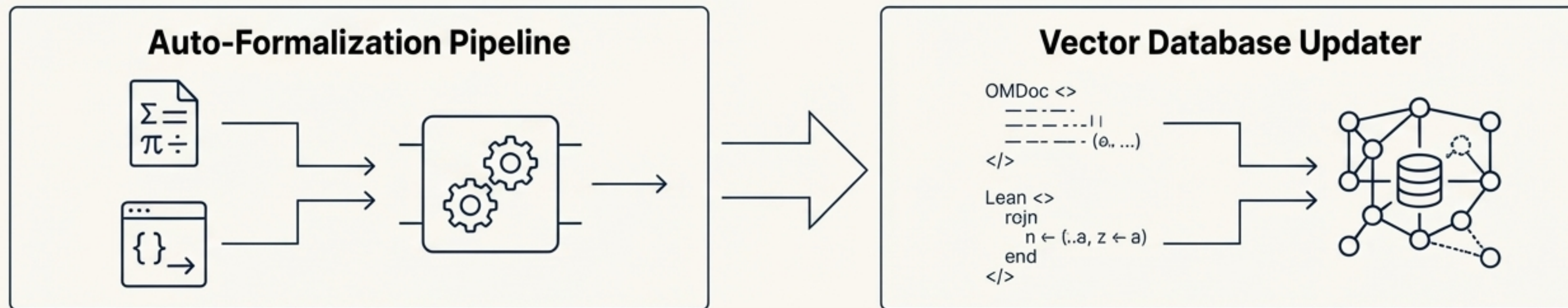
**Action:** It pauses and requests specific guidance from a human mathematician (e.g., 'I am stuck on this lemma; should I apply induction or contradiction?'), acknowledging the need for collaboration in these fields.

# Pillar 5: The Archivist

## Closing the Evolutionary Loop

### Core Objective

Ensure that new theorems (from the **Explorer**) or algorithms (from **FunSearch**) are permanently integrated into the system's core capabilities.



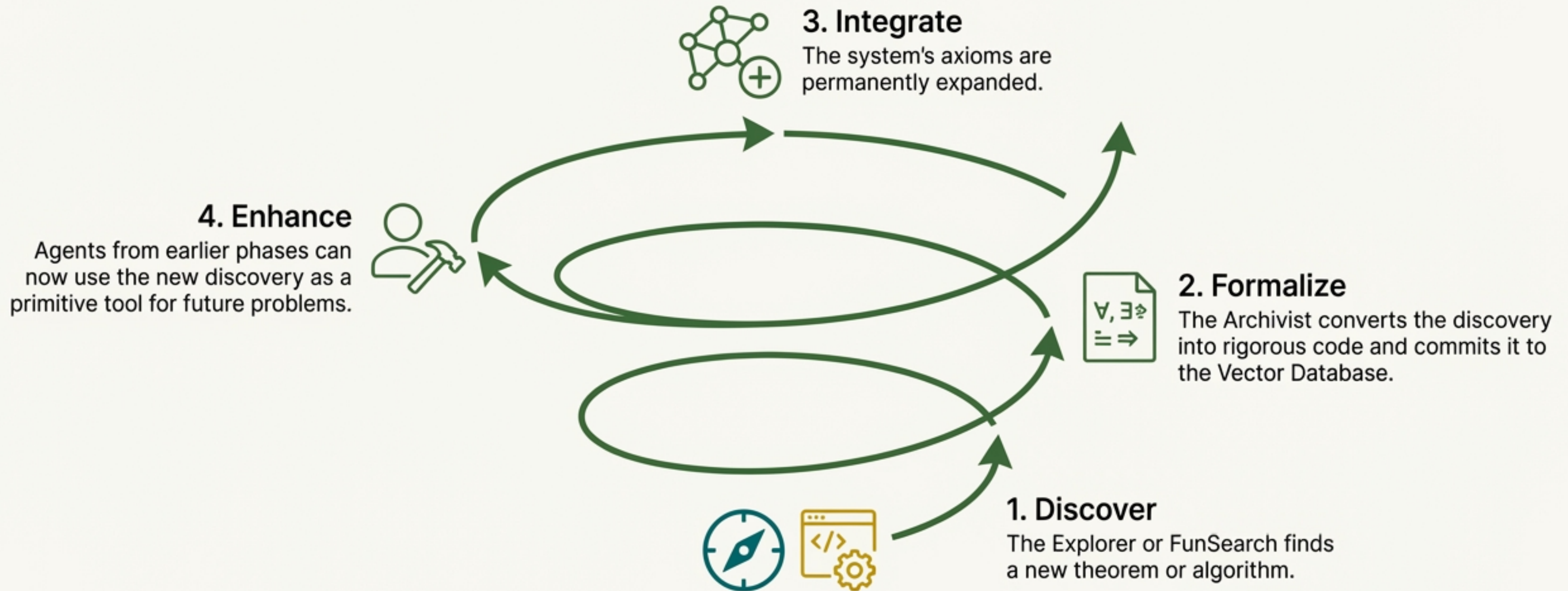
Converts natural language or code outputs into rigorously structured **OMDoc/OpenMath** or **Lean/Isabelle** code.

Re-indexes the system's RAG memory with the new, formalized discoveries.

### The Result

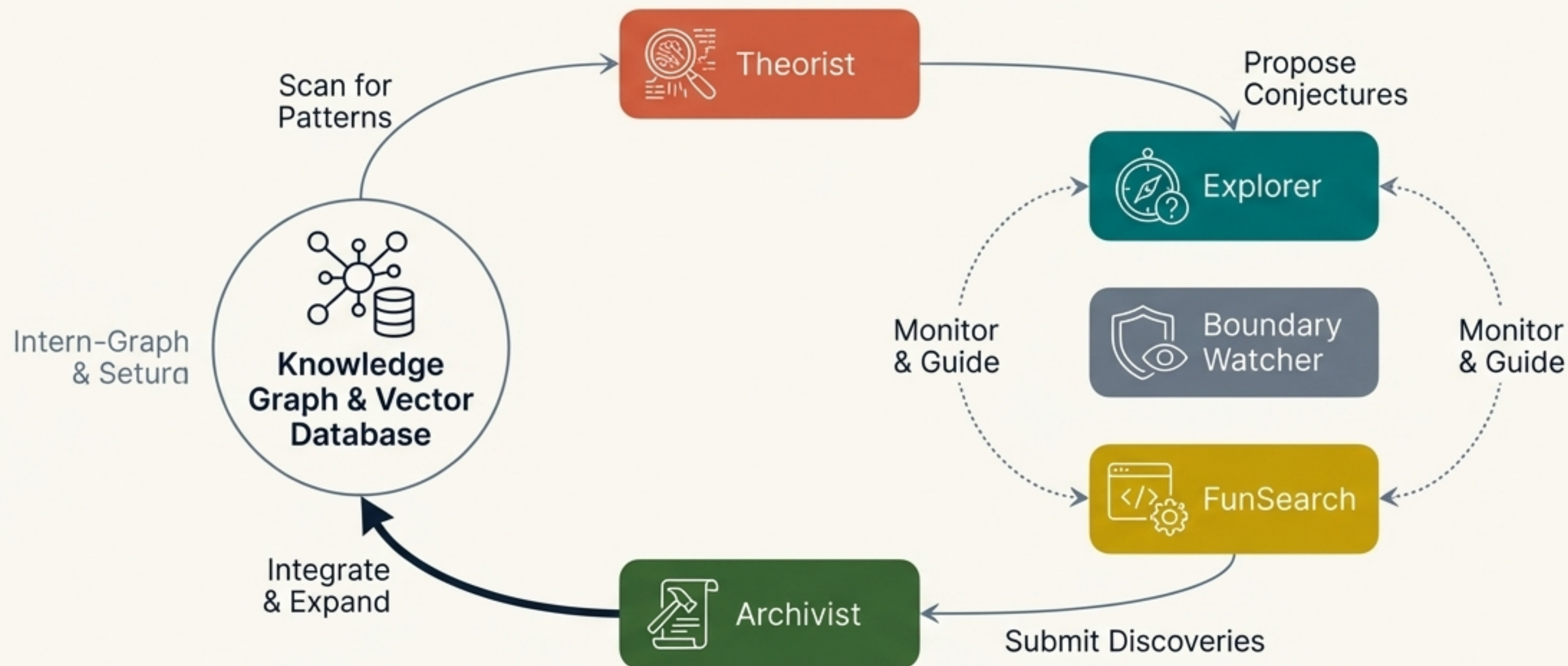
This isn't just data storage; it's a permanent expansion of the system's axioms and available tools.

# The Compounding Intelligence Effect



If the system discovers a new identity for Prime Numbers today, the Algebra Supervisor can utilize that identity as a primitive tool tomorrow.

# The Unified Architecture of the Discovery Engine



The five pillars are not siloed components; they are a deeply integrated system designed for a continuous cycle of discovery and self-improvement.

# The Phase 6 Deliverable: A Junior Research Associate

## Capability Summary



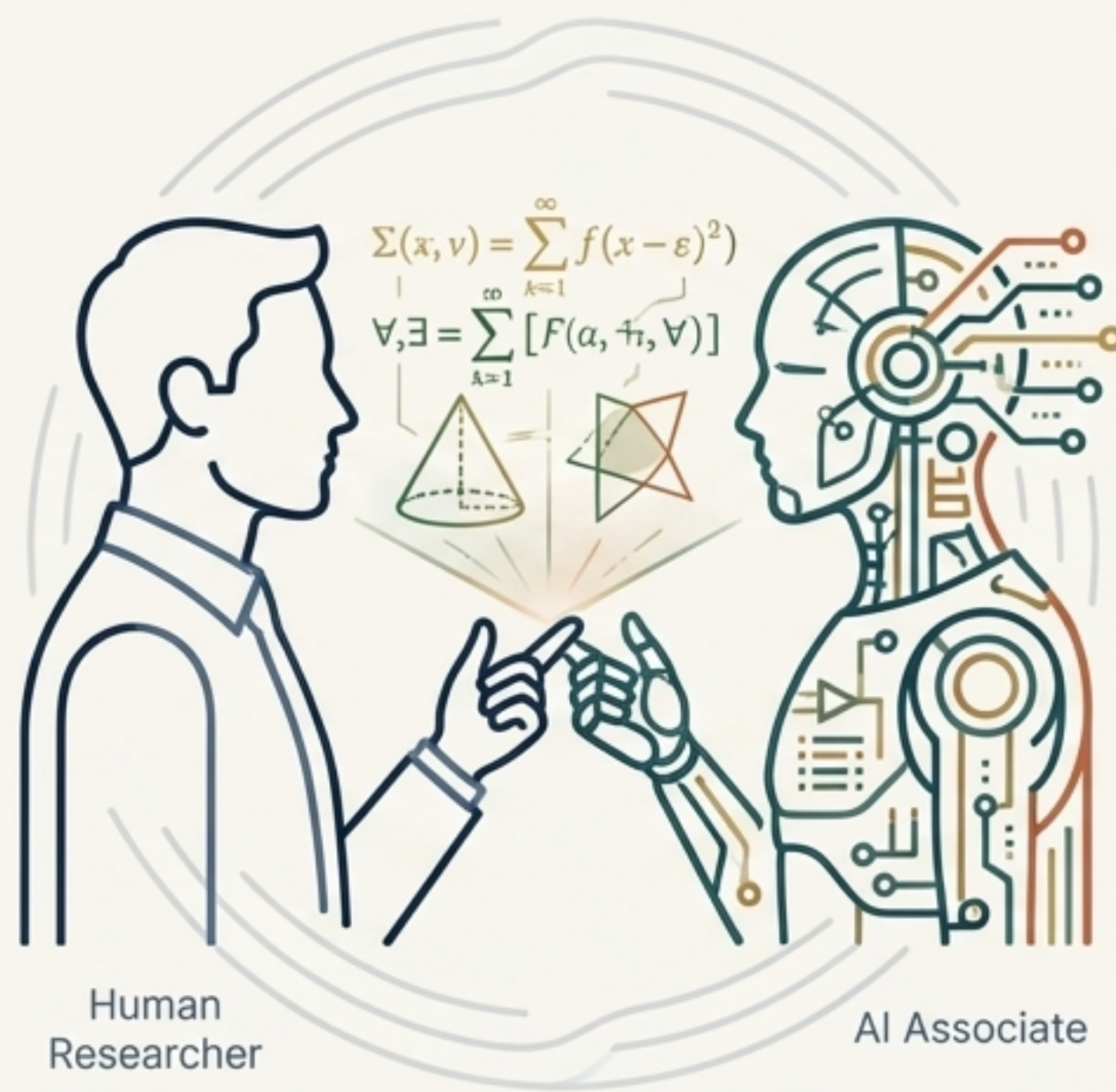
- Can be given an open problem (e.g., 'Find a more efficient sorting algorithm') and run for days.



- Autonomously explores millions of potential code variations (FunSearch) or proof paths (AlphaProof).



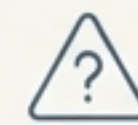
- Formally verifies every step, reporting only mathematically truthful discoveries.



## Safety & Trust



- Recognizes the boundaries of its own logic.



- Flags undecidable problems rather than hallucinating solutions.



- Ensures it remains a trusted, rigorous, and powerful tool for the future of mathematical inquiry.